

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

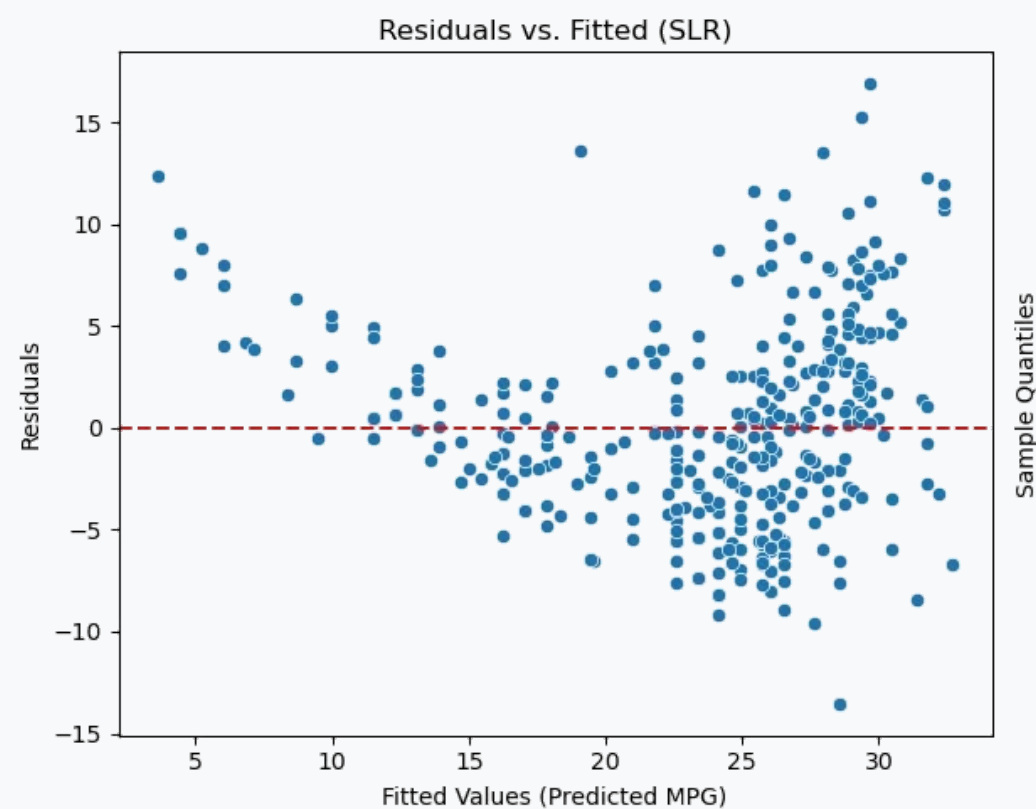
Extending Regression Models

Interactions, curvature, and a regression that predicts three classes

Problems with Simple Linear Models

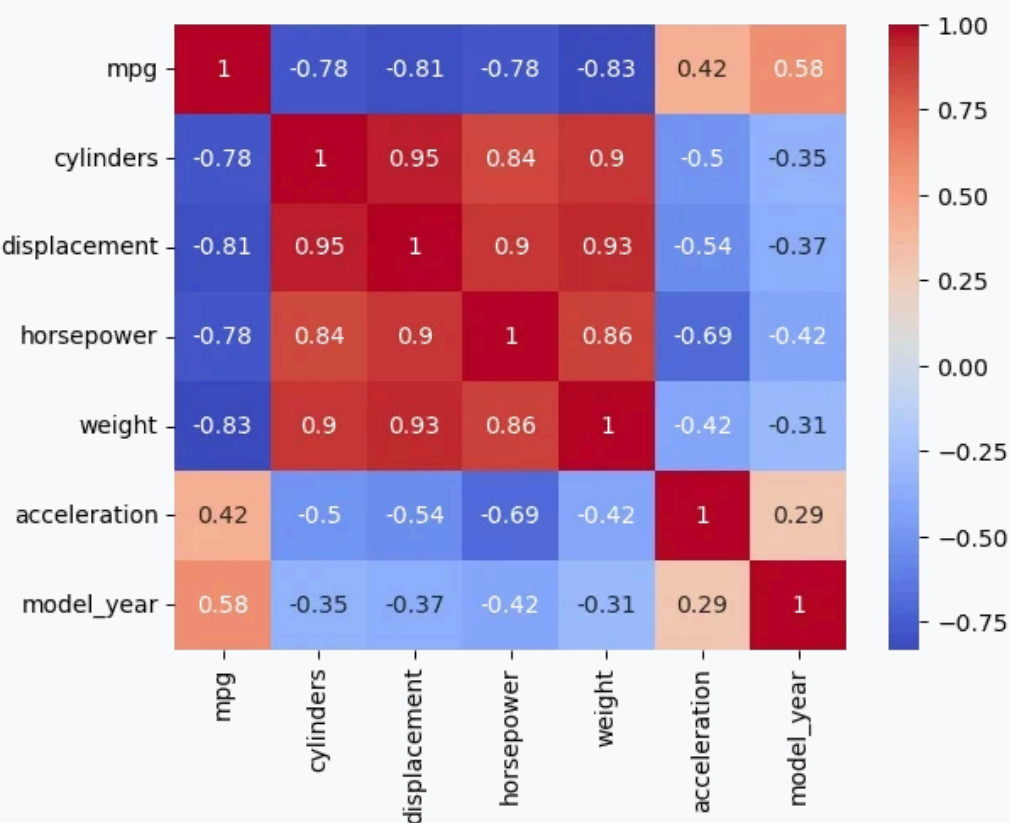
1. Non-Linear Patterns

When modeling `mpg ~ horsepower`, residual plots revealed a clear U-shaped pattern. This indicates **underfitting** (high bias) because the real-world relationship isn't linear.



2. Multicollinearity

Using multiple correlated predictors like horsepower and weight caused unstable p-values and confused interpretation. High correlation between predictors makes coefficient estimates unreliable.



Today we'll fix these problems by extending the linear model, though we'll sacrifice some interpretability in exchange for predictive power.



Interaction Terms: When Effects Depend on Context

Simple linear models assume **additive** effects. But what if the effect of **horsepower** on MPG depends on the car's **weight** ?

Additive Model

$$mpg = \beta_0 + \beta_1 \text{horsepower}$$

Effect of horsepower is constant: β_1

Adj. R-squared: 0.7049

	coef	std err	t	P> t	[0.025	0.975]
Intercept	39.9359	0.717	55.660	0.000	38.525	41.347
horsepower	-0.1578	0.006	-24.489	0.000	-0.171	-0.145

Interaction Model

$$mpg = \beta_0 + \beta_1 \text{horsepower} + \beta_2 \text{weight} + \beta_3 (\text{horsepower} \times \text{weight})$$

Effect varies: $\beta_1 + \beta_3 \times \text{weight}$

Adj. R-squared: 0.7465 ✓

	coef	std err	t	P> t	[0.025	0.975]
Intercept	63.5579	2.343	27.127	0.000	58.951	68.164
horsepower	-0.2508	0.027	-9.195	0.000	-0.304	-0.197
weight	-0.0108	0.001	-13.921	0.000	-0.012	-0.009
horsepower:weight	5.355e-05	6.65e-06	8.054	0.000	4.05e-05	6.66e-05

Interpreting the Interaction Model

How to interpret the model? We can rewrite it as:

$$mpg = \beta_0 + (\beta_1 + \beta_3 \times \text{weight}) \times \text{horsepower} + \beta_2 \text{weight}$$

The effect of **horsepower** is no longer a single number, β_1 . It is now $(\beta_1 + \beta_3 \times \text{weight})$. This means that the effect of **horsepower** **depends on** the **weight** of the car.

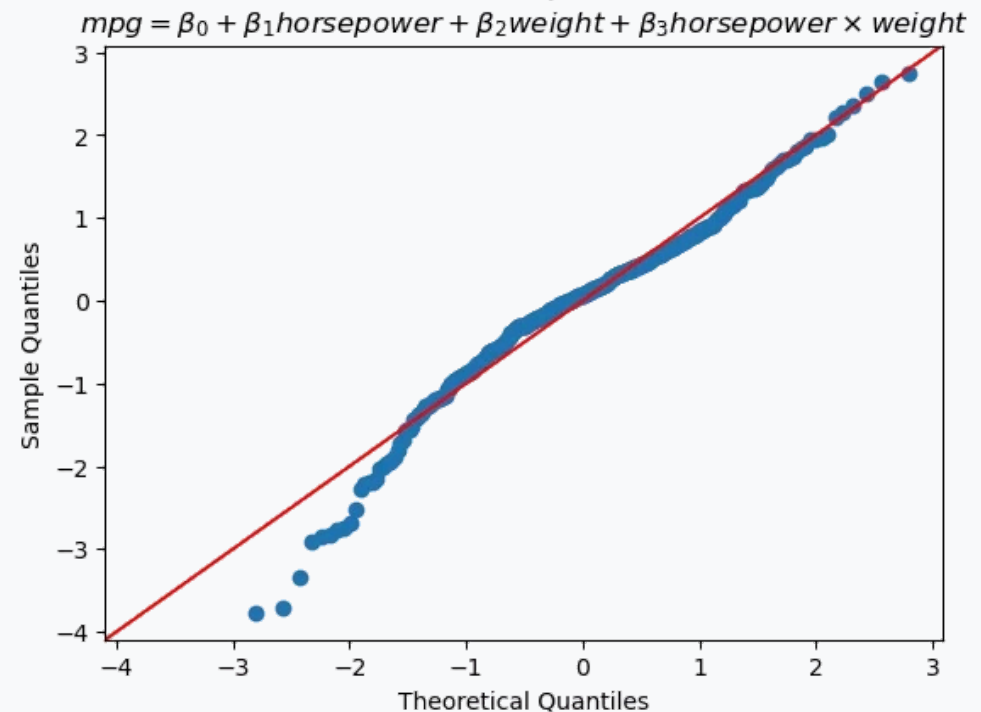
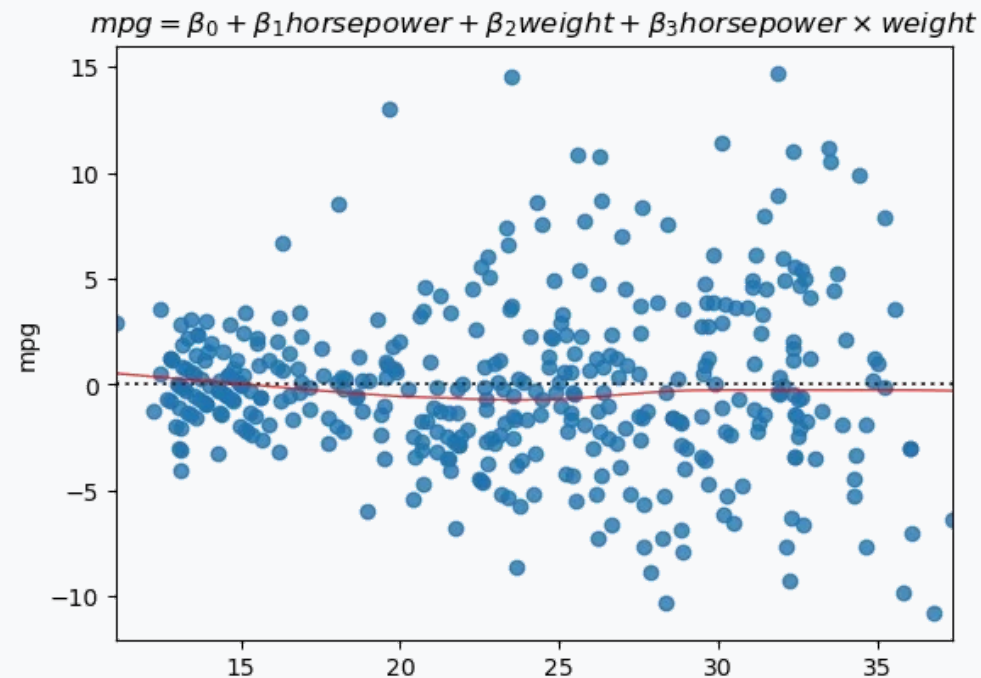
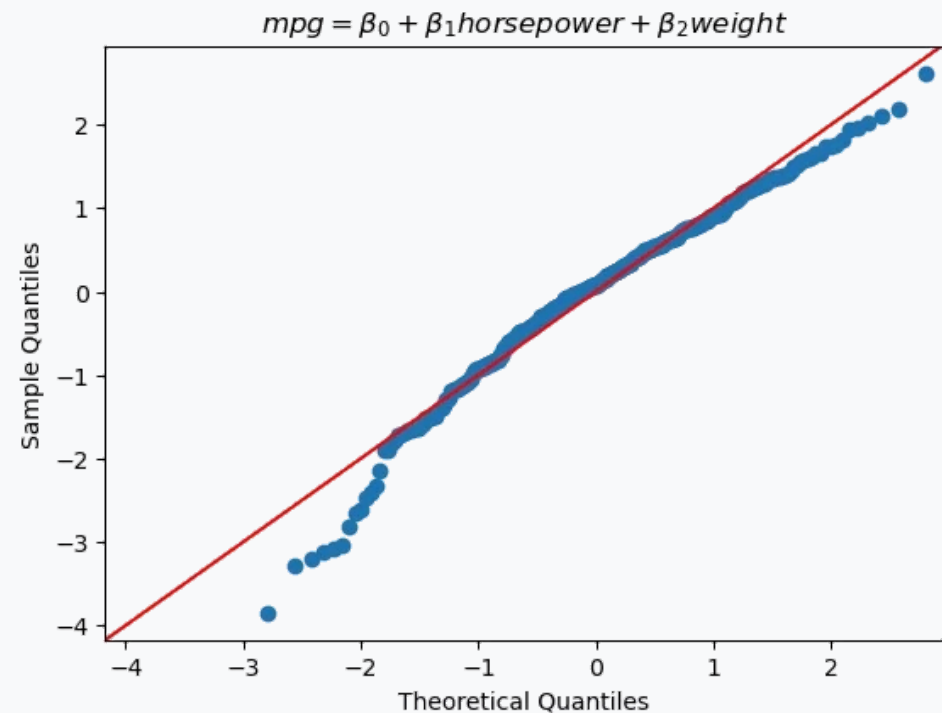
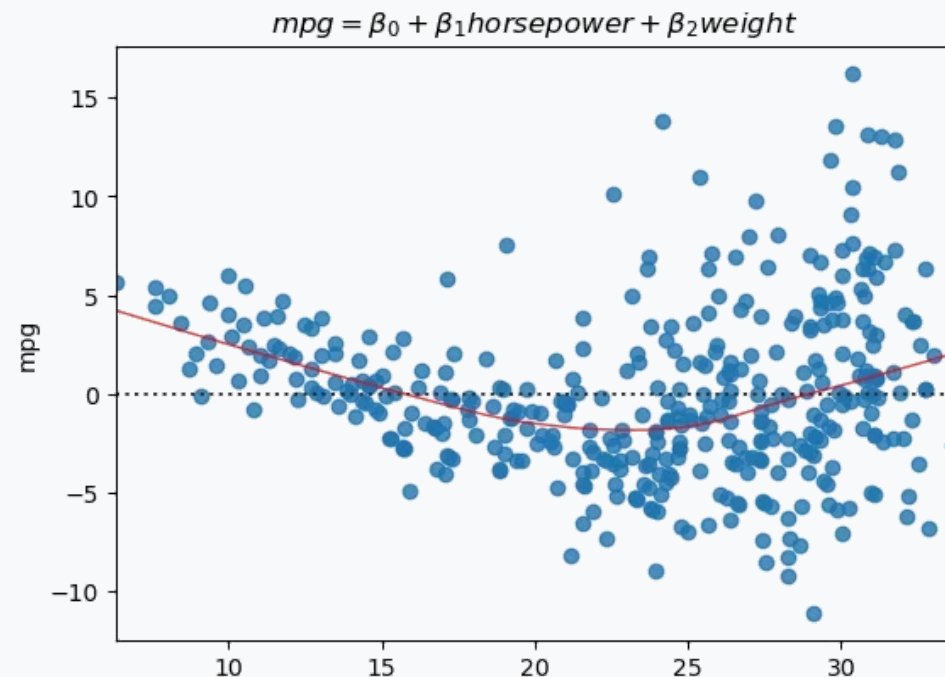
In practice, **when we increase weight by one unit, we increase the effect of horsepower on mpg by β_3 units**. This means that for heavier cars, **horsepower** has a less negative effect on **mpg**.

This makes sense mechanically: in a very heavy car, horsepower is needed just to move it, so the efficiency penalty is smaller compared to a light car where extra horsepower is "overkill".



Comparing the new Model to a Simple Regressor

The model is a little better!



The p-value for the `horsepower:weight` term is tiny (< 0.001), which confirms the interaction is statistically significant — and the residuals, on the bottom row, are visibly flatter than the additive model's.

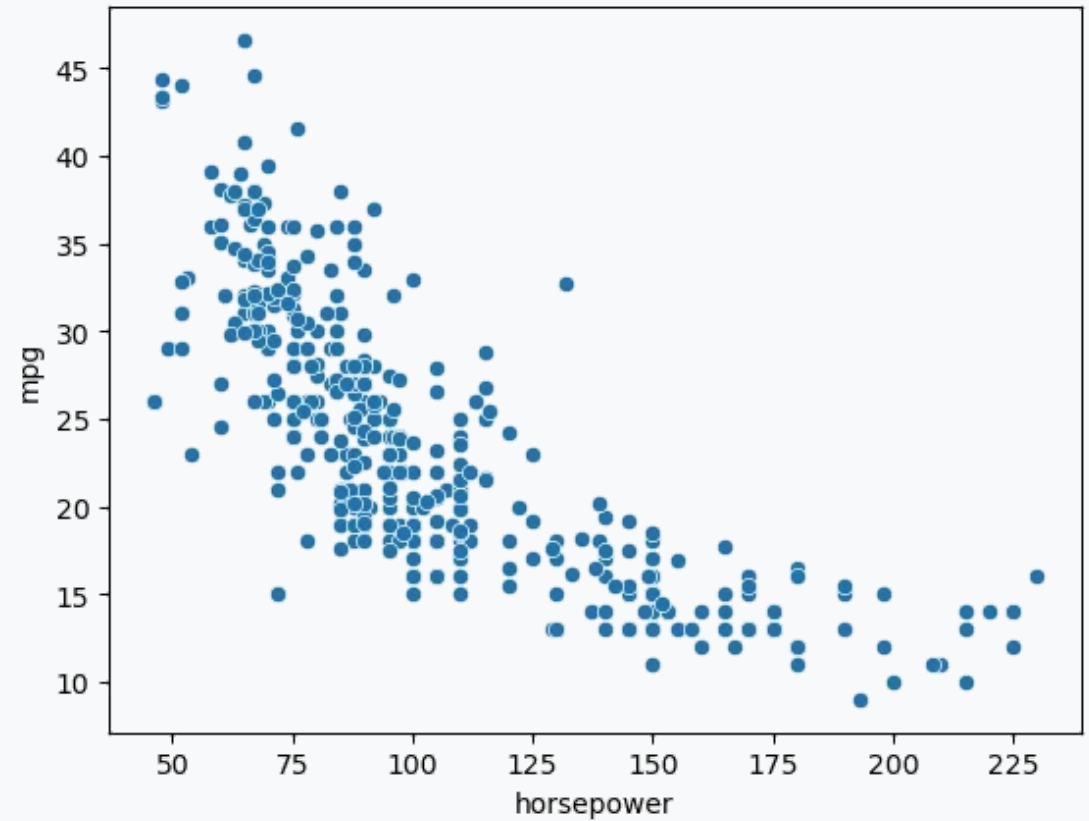


The Non-Linear Reality

The Problem

The relationship between MPG and horsepower isn't linear — it's curved. A straight line simply can't capture this pattern.

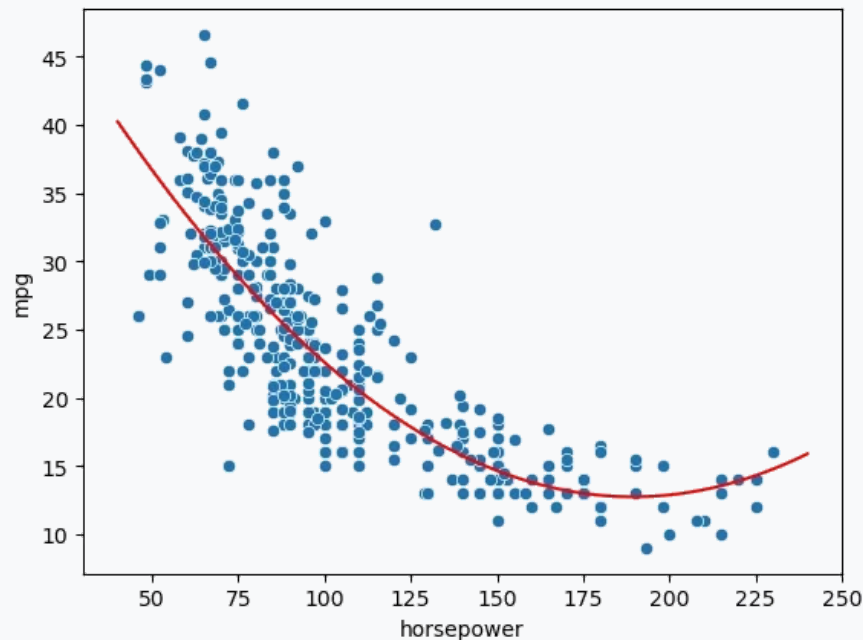
We need a model that can bend and curve to fit the data's natural shape.



Quadratic Regression: Adding Curvature

We can capture the curve by adding a squared term:

$$mpg = \beta_0 + \beta_1 \text{horsepower} + \beta_2 \text{horsepower}^2$$



	coef	std err	t	P> t	[0.025	0.975]
Intercept	56.9001	1.800	31.604	0.000	53.360	60.440
horsepower	-0.4662	0.031	-14.978	0.000	-0.527	-0.405
I(horsepower ** 2)	0.0012	0.000	10.080	0.000	0.001	0.001

This model has a higher R^2 of 0.688, larger than the base 0.606.

We now have the model:

$$y = 56.9001 - 0.4662x + 0.0012x^2$$

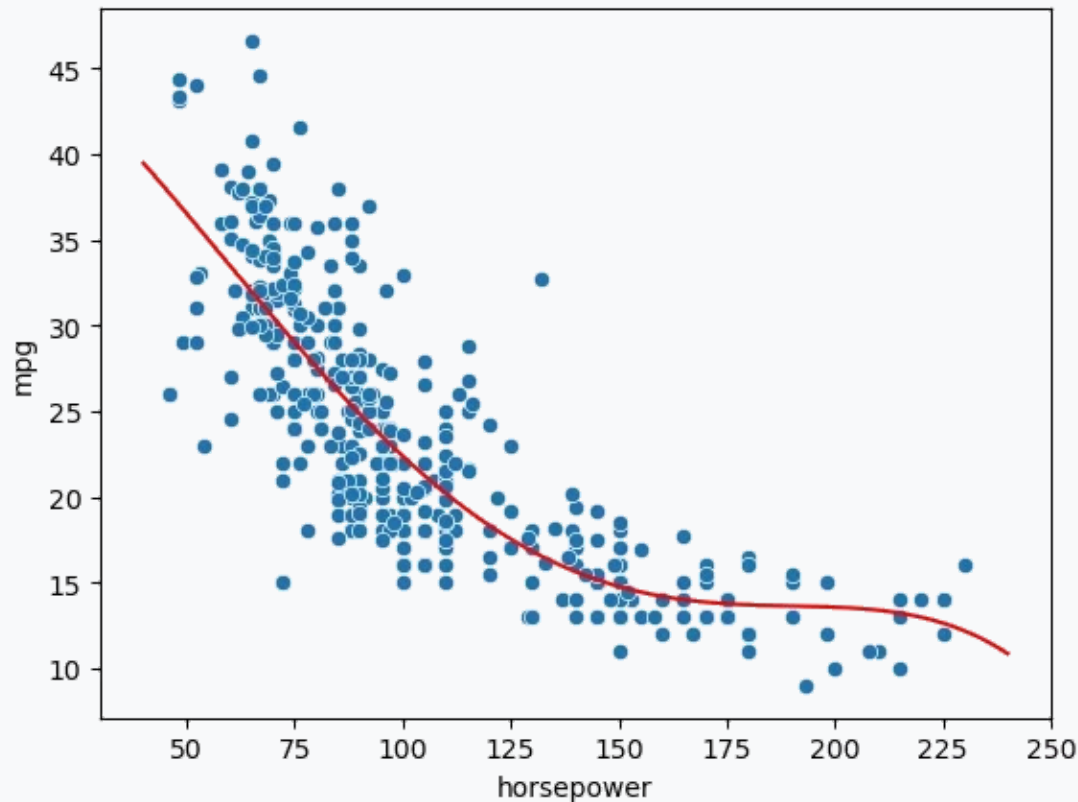
- While we have a better model, we lost interpretability. How do we interpret the coefficient of the quadratic term? How does the interpretation of other coefficients change?



Polynomial Regression: More Flexibility

We can extend this idea to higher-degree polynomials. For degree $d=4$:

$$mpg = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \beta_4x^4$$



Key Insight

While the polynomial function is non-linear in the variable, it's still **linear in its coefficients**. This means we can solve it using ordinary least squares!

We simply create new variables for x^2 , x^3 , etc., and treat it as a linear regression problem.

For instance, this model:

$$y = \beta_0 + \beta_1x + \beta_2y$$

becomes in the case of degree $d = 2$:

$$y = \beta_0 + \beta_1x + \beta_2y + \beta_3x^2 + \beta_4y^2 + \beta_5xy$$

Note that, as the number of variables increases, the number of terms to add for a given degree also increases.



The Diabetes Dataset

In many cases, we may need to study the relationship between some dependent variables and an independent one which is **categorical with more than two possible levels**. Let's consider the diabetes dataset:

	relwt	glufast	glutest	instest	sspg	group
0	0.81	80	356	124	55	Normal
1	0.95	97	289	117	76	Normal
2	0.94	105	319	143	105	Normal
3	1.04	90	356	199	108	Normal
4	1.00	90	323	240	143	Normal

relwt
Relative weight (ratio of actual to expected weight)

glufast
Fasting plasma glucose level

glutest
Glucose intolerance (oral glucose tolerance test area)

instest
Insulin response (insulin area)

sspg
Steady state plasma glucose (measure of insulin resistance)

group
Three diagnostic groups: `Normal` , `Chemical_Diabetic` , or `Overt_Diabetic`



The 145 subjects come from *Data: A Collection of Problems from Many Fields* (Andrews & Herzberg, 1985), through the R package **heplots**.

Multinomial Logistic Regression

The multinomial logistic regression model extends binary logistic regression by modeling the log-odds ratio between each class k and a **baseline class** (typically class 1):

$$\log \left(\frac{P(Y = k | X = \mathbf{x})}{P(Y = 1 | X = \mathbf{x})} \right) = \beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kn}x_n$$

The model estimates $(n+1) \times (K-1)$ **parameters**, where n is the number of predictors and K is the number of classes. Each non-baseline class gets its own set of coefficients.

Through mathematical derivation, we can express the class probabilities as:

Non-baseline classes

$$P(Y = k | X = \mathbf{x}) = \frac{e^{\beta_k^T \mathbf{x}}}{1 + \sum_{l=2}^K e^{\beta_l^T \mathbf{x}}}$$

Baseline class

$$P(Y = 1 | X = \mathbf{x}) = \frac{1}{1 + \sum_{l=2}^K e^{\beta_l^T \mathbf{x}}}$$

These expressions compute class probabilities once parameters are estimated and predictors are observed.

- **Optimisation** We won't see the details, but this is optimised by establishing a similar objective (maximum likelihood or cross-entropy).

- **The softmax parametrisation.** Machine learning writes the same model without a baseline, giving every class its own coefficients: $P(Y = k | \mathbf{x}) = e^{\beta_k^T \mathbf{x}} / \sum_{l=1}^K e^{\beta_l^T \mathbf{x}}$. It is a *reparametrisation*, and an **overparametrised** one — adding the same vector to every β_k changes nothing — which is why a baseline is chosen here and why single coefficients are readable as log-odds ratios only in this form. `scikit-learn` fits the softmax form, `statsmodels` this one. **Not examined.**



Multinomial Logistic Regression Example

If we fit this model with `statsmodels`:

```
group ~ glutest
```

We obtain the following output:

Optimization terminated successfully.
Current function value: 0.100843
Iterations 14

MNLogit Regression Results						
=====						
Dep. Variable:	group	No. Observations:	145			
Model:	MNLogit	Df Residuals:	141			
Method:	MLE	Df Model:	2			
Date:	Sat, 15 Nov 2025	Pseudo R-squ.:	0.9013			
Time:	15:53:12	Log-Likelihood:	-14.622			
converged:	True	LL-Null:	-148.10			
Covariance Type:	nonrobust	LLR p-value:	1.076e-58			
=====						
group=1	coef	std err	z	P> z	[0.025	0.975]

Intercept	-90.3738	42.527	-2.125	0.034	-173.726	-7.022
glutest	0.2153	0.101	2.128	0.033	0.017	0.413

group=2	coef	std err	z	P> z	[0.025	0.975]

Intercept	-109.8266	43.021	-2.553	0.011	-194.147	-25.506
glutest	0.2471	0.102	2.428	0.015	0.048	0.447
=====						

Model Likelihood Ratio Test:
266.9538631863749 1.0757346252010753e-58

Multinomial Logistic Regression Example — cont.

Model Overview

Model Convergence

Optimization terminated successfully after 14 iterations with log-likelihood of -14.622

Likelihood Ratio Test

Statistic: 266.95 with p-value ≈ 0 , showing highly significant improvement over null model

Pseudo R^2

This is $R^2 = 1 - \frac{-14.622}{-148.10} = 0.9013$. The multinomial logistic regressor explains about 90% of the uncertainty in the outcomes.

Two Coefficient Sets

Separate coefficients estimated for `Chemical_Diabetic` and `Overt_Diabetic` groups versus `Normal` baseline

Coefficient Interpretation

1

Chemical Diabetic Intercept: -90.37

When `glutest` is zero, odds of being `Chemical_Diabetic` vs `Normal` are $e^{-90.37} \approx 0$, indicating extremely low probability. The subject is almost certainly `Normal` at baseline.

2

Overt Diabetic Intercept: -109.83

When `glutest` is zero, odds of being `Overt_Diabetic` vs `Normal` are $e^{-109.83} \approx 0$, an even lower probability than `Chemical_Diabetic`.

3

Chemical Diabetic Slope: 0.2153

Each one-unit increase in `glutest` multiplies the odds by $e^{0.2153} \approx 1.24$, representing a **24% increase** in odds of `Chemical_Diabetic` vs `Normal`.

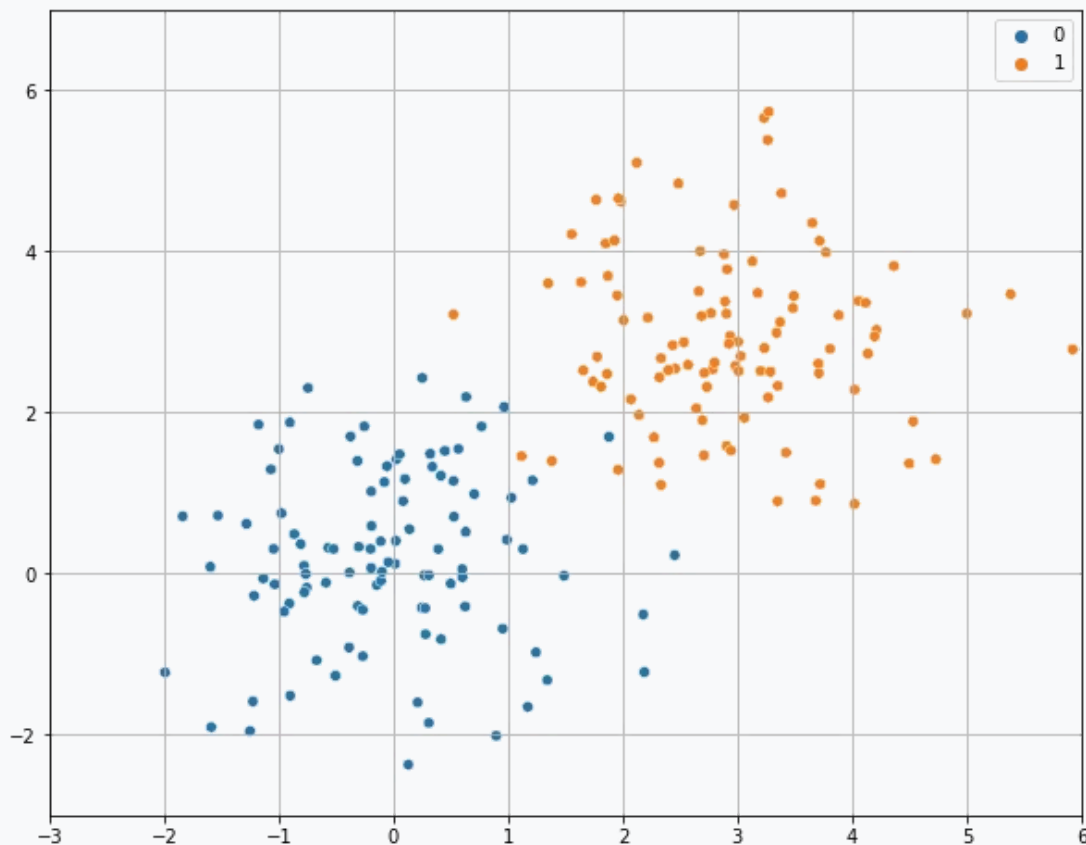
4

Overt Diabetic Slope: 0.2471

Each one-unit increase in `glutest` multiplies the odds by $e^{0.2471} \approx 1.28$, representing a **28% increase** in odds of `Overt_Diabetic` vs `Normal`.



Geometrical Interpretation of the Coefficients of a Logistic Regressor



While the logistic function itself is non-linear, **logistic regression is fundamentally a linear classifier**. The decision boundary — where $P(y=1|x) = 0.5$ — forms a hyperplane in feature space.

The figure on the left shows an example. A linear classifier would find a line separating the two sets of points.

We will see that the logistic regressor does just that.

We define the decision boundary as the set of points of maximum uncertainty, i.e., $P(y = 1|\mathbf{x}) = 0.5$. Recalling that in the case of two dimensions:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2)}}$$

For two-dimensional data, this boundary is hence a straight line defined by:

$$P(y = 1|\mathbf{x}) = 0.5 \Leftrightarrow e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2)} = 1 \Leftrightarrow 0 = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

This can also be rewritten as follows:

$$x_2 = -\frac{\beta_1}{\beta_2} x_1 - \frac{\beta_0}{\beta_2}$$

The coefficients determine both the orientation (slope) and position (intercept) of this separating line (the decision boundary).

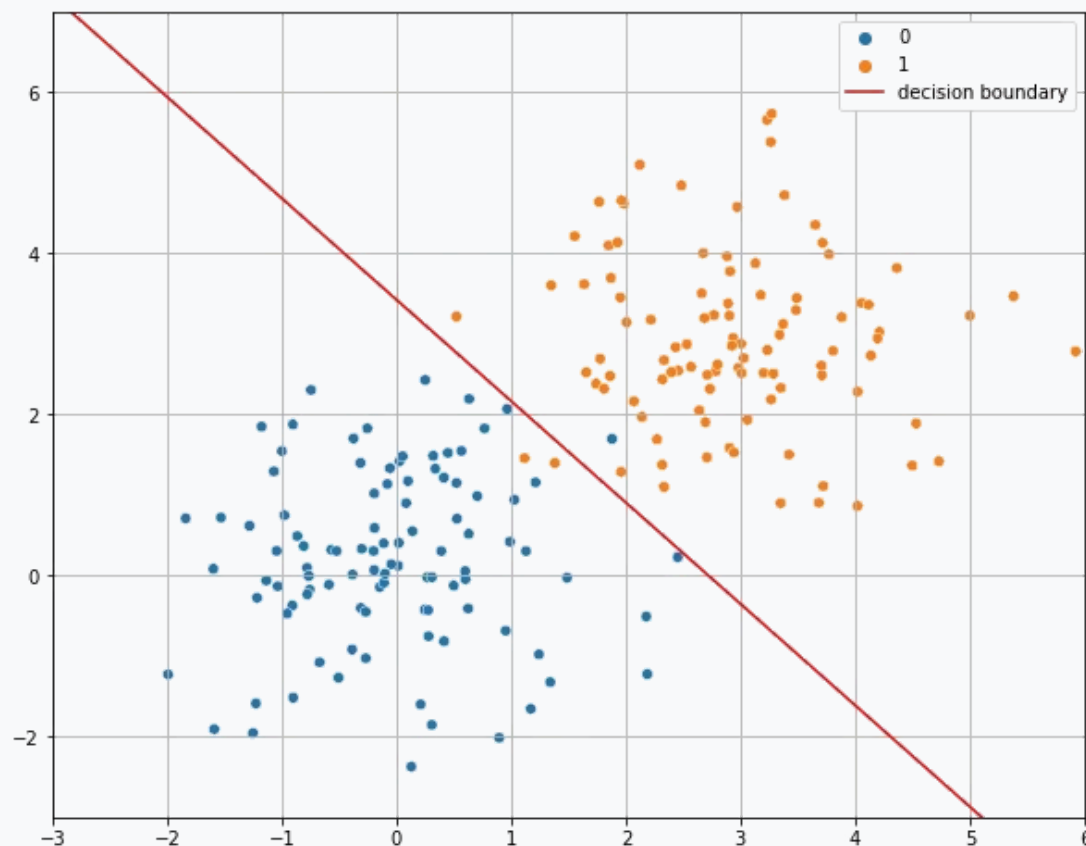
Decision Boundary Example

For example, let's assume we find the following parameters for the logistic regressor $P(y = 1|x_1, x_2) = \sigma(\beta_0 + \beta_1 x_1 + \beta_2 x_2)$:

β_0	β_1	β_2
-3.47	1.17	1.43

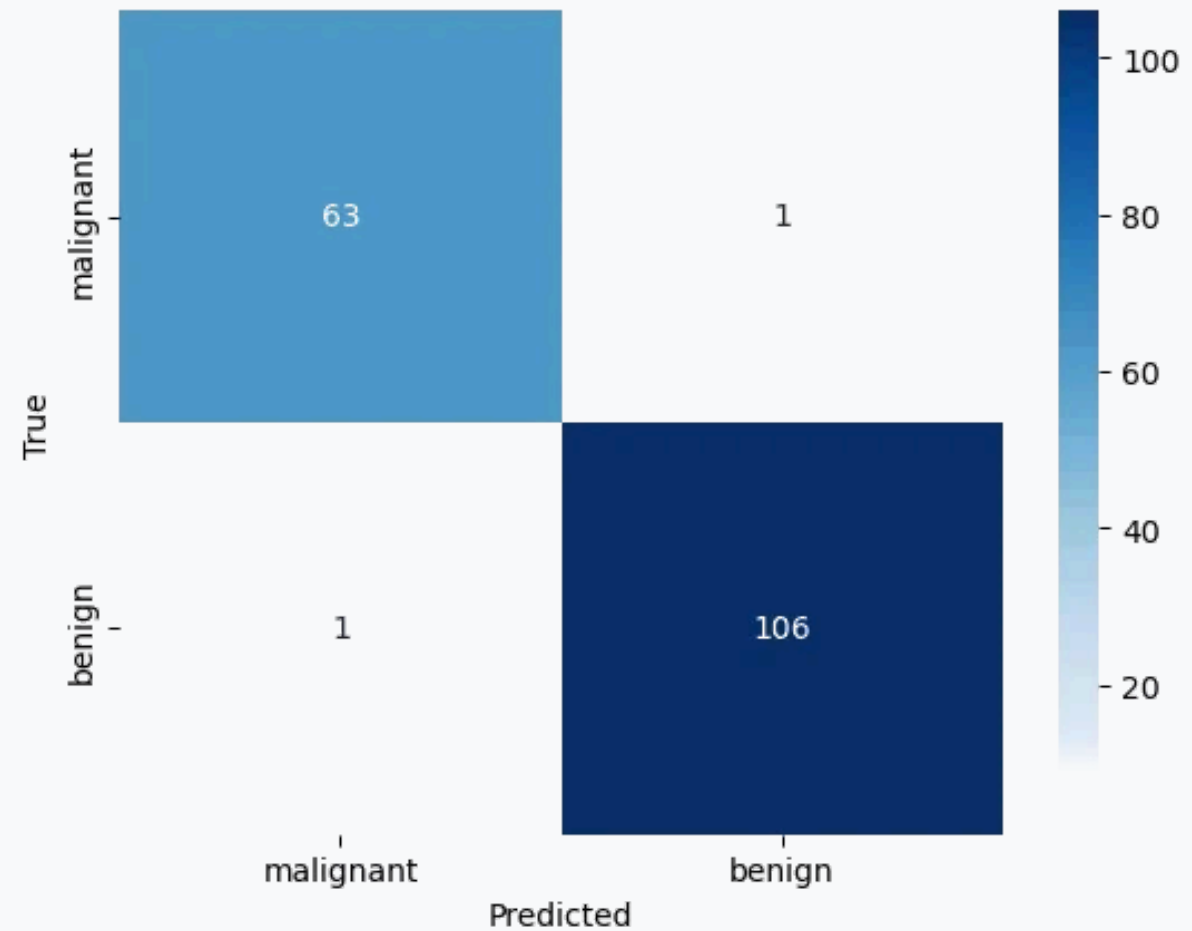
These parameters define the following decision boundary:

$$x_2 = -\frac{1.17}{1.43}x_1 + \frac{3.47}{1.43} = -0.82x_1 + 2.43$$



The line falls from left to right, as a slope of -0.82 requires, and separates the two clouds: below it the model gives the blue class a probability above one half, above it the orange one.

Multiclass Logistic Regression in Python



Conclusions and Next Steps



We Have Explored:

Interaction terms

Quadratic and polynomial regression

Multinomial logistic regression

The geometry of a logistic regressor

- In the next lectures we change perspective: from explaining a relationship to predicting an outcome, and from statistics to machine learning.

References

- Chapter 3 and chapter 4 of: James, Gareth Gareth Michael. *An introduction to statistical learning: with applications in Python*, 2023.
<https://www.statlearning.com>
- Lecture notes: **Chapter 10 — Extending regression models**

